

HW1: Parallel Sparse Matrix-Vector Multiplication (Parallel SpMV)

1. Introduction

Sparse Matrix-Vector Multiplication (SpMV) is a fundamental operation in High-Performance Computing (HPC) and scientific computing. It has significant applications in Static Timing Analysis (STA) and GPU acceleration domains. This assignment focuses on implementing and optimizing the SpMV algorithm, with emphasis on handling large-scale sparse matrix storage, parallel load balancing, and cache performance optimization.

2. Requirements

Implementation Tasks (suggested order)

Follow this incremental implementation order. each step builds on the previous one and helps you observe correctness and performance improvements.

1. Naive COO-scan (**not_csr**) — required first step

- Implement `spmv_not_csr.c` that computes $y = A \cdot x$ by scanning the COO (coordinate) entries for each row (i.e., for each row, iterate through all nonzero entries and accumulate those matching the row).
- Purpose: simple, easy-to-write baseline. Expect poor performance for large matrices; useful to validate correctness and to illustrate why CSR helps.

2. Serial CSR Implementation — second step

- Implement `spmv_serial.c` using the **CSR (Compressed Sparse Row)** format.
- Convert input `.mtx` (Matrix Market) COO entries into CSR and implement an efficient serial SpMV ($y = A \cdot x$).
- Purpose: demonstrate how storage format (CSR) reduces work and memory access compared to the naive COO-scan.

3. Parallel CSR Implementation (OpenMP) — final step

- Implement `spmv_openmp.c` by parallelizing the CSR SpMV with **OpenMP**.
- Explore different scheduling strategies (`static`, `dynamic`, `guided`) and chunk sizes to mitigate load imbalance caused by non-uniform row NNZ distributions.

4. Optimization (Advanced, optional)

- Further optimize for cache behaviour, use blocking, or SIMD (AVX) intrinsics where appropriate.

At each step you should verify correctness (compare with golden) before measuring performance.

Report Analysis Points

- How does matrix size influence performance?

- How does the number of threads influence performance?
- What is the speedup of your parallelized code compared to the serial version?
- Explain your observed scalability: Strong v.s. Weak?
- What optimization techniques did you apply?
- Any additional insights or observations
- How does the choice of storage format (e.g., CSR vs naive COO-scan) influence correctness, memory usage, and runtime performance?

3. Environment

- **OS:** Ubuntu 24.04 Server
- **CPUs:** 144
- **Sockets:** 2
- **Cores per socket:** 36
- **Threads per core:** 2
- **Networking:** All nodes connected via Mellanox ConnectX-5 Adapter to the same switch
- **Total Nodes:** 8

Node Allocation and Restrictions

To accommodate all students, the class is divided into two groups:

- **Group 1 (Team 1 -- Team 25):** Node 0 -- Node 3
- **Group 2 (Team 25 -- Team 50):** Node 4 -- Node 7

Restriction: Each team should not create more than **16 computing processes/threads per node** to minimize interference.

Connection Information (SSH)

- **MPI Nodes IP:** 172.16.179.50 -- 172.16.179.57
- **Jump Server:** 140.112.90.37 (Port: 9037)
 - *NTU CSIE Students:* Can directly log in to MPI Nodes if your client is in the 172.16.0.0/16 subnet. Connect to CSIE VPN first.
 - *Other Students:* Log in to the Jump Server first, then access MPI Nodes from there.

Warning: Do NOT execute computing processes on the Jump Server!

4. Execution

We will run your code as follows (example):

```
# Compilation (example)
gcc -fopenmp -O3 -o spmv_openmp spmv_openmp.c
gcc -O3 -o spmv_serial spmv_serial.c
gcc -O3 -o spmv_not_csr spmv_not_csr.c

# Execution (examples)
# Input files typically contain sparse matrix non-zero elements. The vector
argument is optional; if omitted programs use x=ones.
```

```
# naive COO-scan baseline
./spmv_not_csr input_matrix.mtx [input_vector.txt]

# CSR serial implementation
./spmv_serial input_matrix.mtx [input_vector.txt]

# CSR parallel (OpenMP)
OMP_NUM_THREADS=16 ./spmv_openmp input_matrix.mtx [input_vector.txt]
```

(Note: Specific execution commands will be provided in the sample code)

5. Submission

- **Deadline:** 4/10 (Fri) 23:59
- **Submission Method:** One team member submits a zip file
- **Filename Format:** `Team_<Team Number>_HW1.zip`
- **Required Files:**
- **Required Files:**
 1. `spmv_not_csr.c`: Naive COO-scan implementation
 2. `spmv_serial.c`: CSR-based serial implementation
 3. `spmv_openmp.c`: CSR-based OpenMP parallel implementation
 4. `Team_<Team Number>_HW1_report.pdf`: Project report

Note: Ensure your source code can be compiled and executed without modifications. Deductions will apply if modifications are required.

6. Grading Policy

1. Correctness (10%)

- Points proportional to the number of test cases passed
- Acceptable error margin: ± 0.02
- *If your program produces incorrect answers, you will receive 0% in the Competition Rank section*

2. Scalability (10%)

- Full points if your parallelized code demonstrates good scalability
- Justify scalability in your report

3. Competition Rank (40%)

- Score formula: `score = max(10, 40 × (100 - 5 × (rank - 1)))`
- Programs will be run individually for ranking

4. Report (40%)

- More thorough, reasonable, and clear analysis yields higher scores

Warning:

- Submissions after the deadline will have their total score multiplied by **70%**
- Follow the template code instructions and do not modify code that should not be changed